



Paso de Mensajes



Llamamos **paso de mensajes** a la técnica de comunicación entre procesos que utilizamos cuando **no existe memoria compartida**. Los procesos se comunican **enviando y recibiendo información explícitamente**.

Explicación

Existen sistemas donde **no hay memoria compartida** entre las entidades concurrentes.

En este contexto **no tiene sentido hablar de hebras**, ya que las hebras comparten memoria por definición. Aquí trabajamos con **procesos independientes**, cada uno con su propio espacio de direcciones.

Por este motivo:

- Las técnicas clásicas de concurrencia basadas en memoria compartida (cerrojos, monitores, semáforos sobre variables compartidas, etc.)
NO SON APLICABLES.
- Esto elimina los problemas de **exclusión mutua sobre datos compartidos**.
- Sin embargo, **siguen existiendo problemas de sincronización**, orden de ejecución y bloqueos entre procesos.

Para estos sistemas se utiliza un **modelo distinto de concurrencia**: el **paso de mensajes**.

Comunicación y sincronización en paso de mensajes

Los procesos se comunican exclusivamente mediante:

- **send(msg)** → envío de información.
- **receive(msg)** → recepción de información.

El paso de mensajes **no solo transmite datos**, también **sincroniza** a los procesos.

Por ejemplo:

- Un `send()` **bloqueante** puede actuar como un `wait`.
- Un `receive()` **bloqueante** puede actuar como un `signal`.

Es decir, el propio acto de comunicar introduce **puntos de sincronización** en el flujo del programa.

El envío de mensajes se realiza a través de un **canal**, que es una **abstracción software** que puede representar distintas realidades físicas (memoria, red, sockets, etc.).

Canales

Conceptualmente, un canal suele modelarse como un **buffer** con una capacidad determinada. Tenemos distintos tipos:

Capacidad cero

- No existe buffer.
- Comunicación **síncrona**.
- También llamada **rendezvous**.
- Emisor y receptor deben coincidir **al mismo tiempo** para que el intercambio ocurra.

Capacidad finita

- Existe un buffer con tamaño limitado.
- El `send()` **no se bloquea** hasta que el buffer se llena.
- Cuando el buffer está lleno, el emisor se bloquea.
- Permite desacoplar temporalmente emisor y receptor.

Capacidad ilimitada

- El emisor **nunca se bloquea** al hacer `send()`.
- Modelo idealizado (no realizable físicamente).
- Se usa en teoría para simplificar razonamientos.

Operaciones básicas en paso de mensajes

`send(msg)`

- Envía información al proceso destino (directa o indirectamente).
- El envío es **seguro** cuando se garantiza que el receptor obtiene exactamente el valor que tenía el mensaje en el momento del envío.

`receive(msg)`

- Recibe información desde el canal.
- Puede bloquearse si no hay mensajes disponibles.

A nivel teórico pueden ser síncronos, asíncronos, bloqueantes, y no bloqueantes, pero a nivel práctico en MPJ-Express SIEMPRE SERÁN síncronos bloqueantes.

Sincronía y bloqueo

Las operaciones de envío y recepción pueden clasificarse según dos criterios **independientes**:

- **Sincronía** (relación entre emisor y receptor). Depende del otro.
- **Bloqueo** (comportamiento del proceso local). Depende de mí.

⚠ Muy importante

- **Bloqueante ≠ Síncrono**
- **No bloqueante ≠ Asíncrono**

Sincronía

- **Síncrona**

El intercambio requiere la participación simultánea de emisor y receptor.

- **Asíncrona**

El emisor puede enviar sin que el receptor esté listo; el receptor recogerá el mensaje más tarde (necesitamos un buzón para ello).

Bloqueo

- **Bloqueante**

El proceso se detiene hasta que la operación puede completarse.

- **No bloqueante**

La operación retorna inmediatamente.

¿Cómo identificamos los procesos?

Si vamos a comunicar 2 máquinas entre si debemos identificar los procesos de algún modo.

En sistemas de paso de mensajes es esencial definir **cómo se identifican** los procesos emisores y receptores.

Tenemos distintos tipos.

Denominación directa

- Se utilizan **identificadores explícitos de procesos**.
- Los identificadores se fijan antes de la ejecución.
- Si cambian, es necesario recompilar.
- Adecuada para comunicación **1:1**.
- No es adecuada para modelos cliente-servidor (el servidor no conoce a priori a los clientes).

Denominación indirecta

- Se utiliza un **objeto intermedio**, llamado **buzón**.
- Los procesos envían o reciben mensajes desde el buzón.
- Cuando el buzón permite comunicación **1:n**, se denomina **puerto**.

- Permite relaciones:
 - 1:1
 - 1:n
 - n:m
- Desacopla emisor y receptor y facilita sistemas dinámicos.